CMSIS Cleaning V3 - Concrete Removed Content, Method, and RAG Impact

STM32Cube Preprocessing Pipeline

July 22, 2026

Goal

This document answers, with concrete evidence, what is removed from CMSIS `.h` files by Cleaning V3, how it is removed, and why this improves RAG quality while preserving high-value troubleshooting information.

Evidence Sources

All examples below come from the repository artifacts:

- `data/clean_files_cmsis_device_f3.json` (V2 pre-clean representation)
- `data/clean_files_cmsis_device_f3_v3.json` (V3 cleaned representation)
- `data/chunks_files_cmsis_device_f3_v2.json` (chunk-level concrete macro snippets)
- `pipeline/cleaning/clean_files_v3.py` (cleaning logic)

1) Exactly What Was Removed (Concrete)

A. Repeated legal/banner comment blocks at file top

From `Include/stm32f301x8.h` (V2), the following top block appears and is removed from the retrieval-focused V3 output:

```
/**
 ******************************************************************************
 * @file    stm32f301x8.h
 * @author  MCD Application Team
 * @brief   CMSIS STM32F301x8 Devices Peripheral Access Layer Header File.
 *
 * @attention
 *
 * Copyright (c) 2016 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 ******************************************************************************
 */
```

How removed:

- Regex-based removal in `clean_source_code()` for comment blocks containing Copyright/ST/Licensed/SPD
- Implemented in `pipeline/cleaning/clean_files_v3.py` with `_LICENSE_BLOCK_RE` and `_LICENSE_LINE_R`

B. Massive repeated bitfield macro series (_Pos/_Msk)

From `Include/stm32f301x8.h` chunk evidence (V2):

```
/******************  Bit definition for ADC_ISR register  ******************/
#define ADC_ISR_ADRDY_Pos              (0U)
#define ADC_ISR_ADRDY_Msk              (0x1UL << ADC_ISR_ADRDY_Pos)
#define ADC_ISR_EOSMP_Pos              (1U)
#define ADC_ISR_EOSMP_Msk              (0x1UL << ADC_ISR_EOSMP_Pos)
#define ADC_ISR_EOC_Pos                (2U)
#define ADC_ISR_EOC_Msk                (0x1UL << ADC_ISR_EOC_Pos)
...
```

How removed:

- For giant CMSIS headers, V3 does not keep arbitrary `#define` macro banks.
- V3 reconstructs a compact representation from 3 kept sections only: IRQ enum, base addresses, and `_TypeDef` register structs.
- This behavior is explicit in `filter_cmsis_device_header()` docstring: drops `XXX_Pos` / `XXX_Msk` lines.

C. Other generated repeated macro templates

Example classes removed with the same rule include repeated GPIO macro families:

```
#define GPIO_MODER_MODER3_Pos          (6U)
#define GPIO_MODER_MODER3_Msk          (0x3UL << GPIO_MODER_MODER3_Pos)
#define GPIO_MODER_MODER3_0            (0x1UL << GPIO_MODER_MODER3_Pos)
#define GPIO_MODER_MODER3_1            (0x2UL << GPIO_MODER_MODER3_Pos)
...
```

2) What Is Kept (to avoid losing important info)

For the same file in V3, the kept summary states:

```
/* CMSIS Device Header - Filtered for RAG relevance */
/* Original: 698,576 chars, 6,565 #defines */
/* Kept: IRQ enum, 63 base addresses, 22 peripheral structs */
```

Kept content includes:

- IRQ vectors: interrupt names and numbers used in debugging and fault triage.
- Base addresses: `PERIPH_BASE`, `GPIOA_BASE`, etc.
- Register layouts: `typedef struct ... _TypeDef;` used for register map understanding.

3) Quantified Impact

Using values embedded in V3 artifacts:

Header	Original chars	Cleaned chars	Reduction	Kept signal
`Include/stm32f301x8.h`	698,576	27,567	96.05%	IRQ + 63 BASE + 22 structs

Header	Original chars	Cleaned chars	Reduction	Kept signal
`Include/stm32f302x8.h`	1,040,619	29,498	97.17%	IRQ + 65 BASE + 25 structs

4) RAG Value and Why This Is Defensible

RAG effect:

- Much denser technical evidence per token.
- Fewer false lexical matches caused by repetitive macro banks.
- Better context budget usage for question-relevant signals.

Not losing critical information:

- High-value troubleshooting anchors are preserved (IRQ/base/register layout).
- Original full source remains reachable via `github_url` for deep bit-level investigations.
- If strict bitfield recall is required, a fallback path can query full headers on demand.

5) One-line Defense Statement (for Jury)

Cleaning V3 does not delete project knowledge; it removes repetitive retrieval noise and keeps the highest-value hardware evidence for faster, more reliable RAG answers.

Appendix: Exact Cleaning Logic

Implemented in:

- `pipeline/cleaning/clean_files_v3.py`

Key rules:

- `clean_source_code()` removes legal/banner boilerplate with regex patterns.
- `is_cmsis_device_header()` activates giant-header filtering (size threshold 100KB).
- `filter_cmsis_device_header()` keeps IRQ/base/typedef and drops massive `_Pos/_Msk` banks.